

## 1. Axios kya hai?

Axios ek third-party HTTP client library hai jo APIs ko request bhejne aur response receive karne ke liye use hoti hai.

```
JavaScript
  ↓
Axios
  ↓
Third-party API
  ↓
Server
  ↓
Response
  ↓
Axios
  ↓
JavaScript
```

Install:

```
</> Bash
npm install axios
```



Browser me CDN se bhi use kar sakte ho, lekin Node/React project me npm common hai.

| Feature                         | fetch                                 | Axios             |
|---------------------------------|---------------------------------------|-------------------|
| HTTP request                    | ✓                                     | ✓                 |
| Promise                         | ✓                                     | ✓                 |
| async/await                     | ✓                                     | ✓                 |
| JSON response handling          | manually <code>response.json()</code> | automatic parsing |
| Request config                  | ✓                                     | ✓                 |
| Interceptors                    | ✗ native                              | ✓                 |
| Request timeout                 | less convenient                       | ✓                 |
| Error handling                  | thoda manual                          | convenient        |
| Request/response transformation | limited                               | ✓                 |

### 3. Sabse basic Axios request

⌞ JavaScript

```
const axios = require("axios");

axios.get("https://example.com/api/users")
  .then(response => {
    console.log(response.data);
  })
  .catch(error => {
    console.log(error);
  });
```

Yahan:

⌞ JavaScript

```
axios.get(...)
```

Promise return karta hai.

## 4. Axios + async/await

Modern code:

```
</> JavaScript

const axios = require("axios");

async function getUsers() {

  try {

    const response = await axios.get(
      "https://example.com/api/users"
    );

    console.log(response.data);

  } catch (error) {

    console.log(error);

  }
}
```

## 5. Axios me `response.data` kya hai?

Ye `fetch()` aur Axios ka important difference hai.

### Fetch:

`</>` JavaScript

```
const response = await fetch(url);  
  
const data = await response.json();
```

### Axios:

`</>` JavaScript

```
const response = await axios.get(url);  
  
const data = response.data;
```

Axios JSON response ko normally automatically parse kar deta hai.

```
import axios from "axios";

async function getProducts() {

  try {

    const response = await axios.get(
      "https://api.example.com/products"
    );

    console.log(response.data);

  } catch (error) {

    console.log(error);

  }
}
```

## 7. API response me kya aa sakta hai?

Suppose third-party API:

`</>` JSON

```
{
  "id": 101,
  "title": "Laptop",
  "price": 50000,
  "description": "Gaming laptop",
  "image": "https://example.com/laptop.jpg",
  "rating": 4.5
}
```

Axios:

`</>` JavaScript

```
const response = await axios.get(url);

console.log(response.data);
```



Then:

`</>` JavaScript

```
console.log(response.data.title);
console.log(response.data.price);
console.log(response.data.image);
console.log(response.data.rating);
```

Tum **data** + **image URL** + **price** + **rating** etc. use kar sakte ho.

## 8. API se image display karna

Suppose:

**</> JavaScript**

```
const response = await axios.get(url);  
  
const product = response.data;
```

HTML:

**HTML**

```
<img id="productImage">
```

JavaScript:

**</> JavaScript**

```
document.querySelector("#productImage").src =  
    product.image;
```



## 9. Multiple products

API response:

`</>` JSON

```
[
  {
    "title": "Laptop",
    "price": 50000,
    "image": "laptop.jpg"
  },
  {
    "title": "Phone",
    "price": 25000,
    "image": "phone.jpg"
  }
]
```

</> JavaScript

```
async function getProducts() {  
  
  try {  
  
    const response = await axios.get(  
      "https://example.com/api/products"  
    );  
  
    const products = response.data;  
  
    products.forEach(product => {  
  
      console.log(product.title);  
      console.log(product.price);  
      console.log(product.image);  
  
    });  
  
  } catch (error) {  
  
    console.log(error);  
  
  }  
}
```



## 10. Axios GET

Data lena:

⌘ JavaScript

```
const response = await axios.get(url);
```

Example:

⌘ JavaScript

```
const response = await axios.get(
  "https://example.com/api/users"
);
```

---

## 11. Axios POST

Data bhejna:

⌘ JavaScript

```
const response = await axios.post(
  "https://example.com/api/users",
  {
    name: "Tarun",
    age: 22
  }
);
```

`</>` JavaScript

```
async function createUser() {  
  
  try {  
  
    const response = await axios.post(  
      "https://example.com/api/users",  
      {  
        name: "Tarun",  
        age: 22  
      }  
    );  
  
    console.log(response.data);  
  
  } catch (error) {  
  
    console.log(error);  
  
  }  
}
```



## 12. Axios PUT

Complete object update/replace:

`</>` JavaScript

```
await axios.put(  
  "https://example.com/api/users/10",  
  {  
    name: "Tarun",  
    age: 23  
  }  
);
```

## 13. Axios PATCH

Partial update:

```
</> JavaScript

await axios.patch(
  "https://example.com/api/users/10",
  {
    age: 23
  }
);
```

## 14. Axios DELETE

```
</> JavaScript

await axios.delete(
  "https://example.com/api/users/10"
);
```

## 15. Query parameters

Suppose API:

```
/api/products?category=mobile&page=2
```

Axios me:

**</> JavaScript**

```
const response = await axios.get(
  "https://example.com/api/products",
  {
    params: {
      category: "mobile",
      page: 2
    }
  }
);
```

Axios URL automatically bana dega:

```
/api/products?category=mobile&page=2
```

Ye hardcoded URL banane se cleaner hai.

## 16. Headers

Third-party API ko headers dene pad sakte hain:

**</> JavaScript**

```
const response = await axios.get(url, {

  headers: {
    Authorization: "Bearer YOUR_TOKEN",
    "Content-Type": "application/json"
  }

});
```

## 17. API Key

Kuch APIs API key maangti hain:

`</>` JavaScript

```
const response = await axios.get(url, {  
  
  params: {  
    apiKey: "YOUR_API_KEY"  
  }  
  
});
```

Ya header:

`</>` JavaScript



```
const response = await axios.get(url, {  
  
  headers: {  
    "X-API-Key": "YOUR_API_KEY"  
  }  
  
});
```

Secret API key ko frontend me expose nahi karna chahiye agar provider use secret rakhta hai.

## 18. Axios error handling

```
</> JavaScript

try {

    const response = await axios.get(url);

    console.log(response.data);

} catch (error) {

    console.log(error.message);

}
```

Axios error me useful information ho sakti hai:

Axios error me useful information ho sakti hai:

**</> JavaScript**

```
catch (error) {  
  
    if (error.response) {  
  
        console.log(error.response.status);  
        console.log(error.response.data);  
  
    } else if (error.request) {  
  
        console.log("Server se response nahi mila");  
  
    } else {  
  
        console.log(error.message);  
    }  
  
}
```

## 19. Promise chaining with Axios

Async/await se pehle Promise chaining:

**</> JavaScript**

```
axios.get("/api/users")
  .then(response => {

    return axios.get(
      `/api/orders/${response.data.id}`
    );

  })
  .then(response => {

    console.log(response.data);

  })
  .catch(error => {

    console.log(error);

  });
```



## 20. Same thing async/await

</> JavaScript

```
async function getUserOrders() {  
  
  try {  
  
    const userResponse =  
      await axios.get("/api/users/10");  
  
    const orderResponse =  
      await axios.get(  
        `/api/orders/${userResponse.data.id}`  
      );  
  
    console.log(orderResponse.data);  
  
  } catch (error) {  
  
    console.log(error);  
  
  }  
}
```



## 21. Parallel API requests

Agar APIs ek doosre par dependent nahi hain:

JavaScript

```
const [users, products] = await Promise.all([

  axios.get("/api/users"),

  axios.get("/api/products")

]);

console.log(users.data);
console.log(products.data);
```

## 22. Axios Instance

Large project me baar-baar:

JavaScript

```
axios.get("https://api.example.com/...")
```

likhna inconvenient ho sakta hai.

Instance:

JavaScript

```
const api = axios.create({
  baseURL: "https://api.example.com"
});
```

Ab:

```
</> JavaScript  
  
const response = await api.get("/users");
```

instead of:

```
</> JavaScript  
  
axios.get("https://api.example.com/users");
```

---

## 23. Interceptors — Axios ka powerful feature

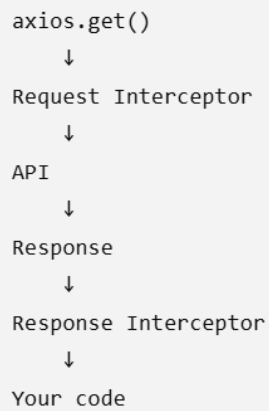
Request API ko jaane se pehle kuch common kaam karna ho:

```
</> JavaScript  
  
api.interceptors.request.use(config => {  
  
    config.headers.Authorization =  
        `Bearer ${token}`;  
  
    return config;  
});
```

Ab har request me automatically token attach kiya ja sakta hai.

Flow:

```
axios.get()
  ↓
Request Interceptor
  ↓
API
  ↓
Response
  ↓
Response Interceptor
  ↓
Your code
```



Interceptors authentication, logging, common headers, centralized response handling etc. me useful hote hain.

## 24. Timeout

`</>` JavaScript

```
const api = axios.create({
  baseURL: "https://api.example.com",
  timeout: 5000
});
```

Agar request unnecessarily long ho jaye, Axios timeout ke baad error de sakta hai.

## 25. Axios + React ka typical flow

React Component



useEffect()



Axios



Third-party API



Promise



await



response.data



setState()



React re-render



UI



```
useEffect(() => {  
  
  async function loadProducts() {  
  
    try {  
  
      const response =  
        await axios.get(  
          "https://example.com/api/products"  
        );  
  
      setProducts(response.data);  
  
    } catch (error) {  
  
      console.log(error);  
  
    }  
  
  }  
  
  loadProducts();  
}
```



## 26. Axios vs Fetch — sabse important

### Fetch:

```
</> JavaScript

const response = await fetch(url);

if (!response.ok) {
  throw new Error("Request failed");
}

const data = await response.json();
```

### Axios:

```
</> JavaScript

const response = await axios.get(url);

const data = response.data;
```



Axios ka syntax generally shorter/convenient hai.

Haan, tum Node.js ke built-in `http` module se API call aur response `res.write()` se bhejne ki baat kar rahe ho. Ye Axios/Express se pehle ka fundamental concept hai.

### 1. `http.get(url, callback)` kya hai?

Node.js ka built-in `http` module external HTTP server/API ko **GET request** bhej sakta hai.

```
</> JavaScript

const http = require("http");

http.get("http://example.com", (res) => {

  console.log(res.statusCode);

});
```



Yahan:

```
</> JavaScript  
http.get(url, callback)
```

ka matlab:

URL par GET request bhejo, aur jab response aaye to callback execute karo.

## 2. `res` kya hai?

Callback ke andar:

```
</> JavaScript  
(res) => {}
```

ye `res` server ka response object hai.

Isme information aa sakti hai:

```
</> JavaScript  
res.statusCode  
res.headers  
res.on("data", ...)  
res.on("end", ...)
```

Important: yahan `res` ko Express ke `res` se confuse mat karo.

### 3. API response chunks me kyu aata hai?

Network se data ek hi baar me zaroori nahi aata.

Server



Chunk 1



Chunk 2



Chunk 3



Chunk 4

Isliye:

`</>` JavaScript

```
res.on("data", chunk => {  
  console.log(chunk);  
});
```

use karte hain.



## 4. res.on("data")

```
</> JavaScript

const http = require("http");

http.get("http://example.com", (res) => {

  res.on("data", (chunk) => {
    console.log(chunk);
  });

});
```

`chunk` usually **Buffer** hota hai.

String me convert:

```
</> JavaScript

res.on("data", (chunk) => {
  console.log(chunk.toString());
});
```

## 5. `res.on("end")`

Jab poora response receive ho jata hai:

**</> JavaScript**

```
res.on("end", () => {  
    console.log("Response complete");  
});
```

Complete pattern:

**</> JavaScript**

```
const http = require("http");  
  
http.get("http://example.com", (res) => {  
  
    let data = "";  
  
    res.on("data", (chunk) => {  
        data += chunk;  
    });  
});
```



## 6. Third-party JSON API

Suppose API JSON return karti hai:

JSON

```
{  
  "name": "Tarun",  
  "age": 22  
}
```

Node.js:

JavaScript

```
const http = require("http");  
  
http.get("http://example.com/api/user", (res) => {  
  
  let data = "";  
  
  res.on("data", (chunk) => {  
    data += chunk;  
  });  
});
```



JavaScript

```
let data = "";

res.on("data", (chunk) => {
  data += chunk;
});

res.on("end", () => {

  const user = JSON.parse(data);

  console.log(user.name);
  console.log(user.age);

});

});
```

## 7. Ab `res.write()` kaha aata hai?

Ab important distinction:

`http.get()` ka `res`

`</> JavaScript`

```
http.get(url, (res) => {});
```



Ye **third-party server** se aaya response hai.

`res.write()`

Usually Node.js me jab **tum khud server bana rahe ho**, tab client ko response bhejne ke liye:

`</> JavaScript`

```
res.write()
```



use karte ho.



```
const http = require("http");

const server = http.createServer((req, res) => {

  res.write("Hello from server");

  res.end();

});

server.listen(5000);
```

Browser:

http://localhost:5000

Response:

Hello from server



```
const http = require("http");

const server = http.createServer((req, res) => {

  http.get("http://example.com/api/users", (apiRes) => {

    let data = "";

    apiRes.on("data", (chunk) => {
      data += chunk;
    });

    apiRes.on("end", () => {

      res.write(data);

      res.end();

    });

  });

});
```



Yahan do `res` concepts hain, isliye naam clear rakhna better hai.

`</>` **JavaScript**

```
http.createServer((req, clientRes) => {

  http.get(url, (apiRes) => {

  });

});
```

Ab:

```
clientRes = tumhare browser ko response

apiRes = third-party API ka response
```

## 9. Best naming

</> JavaScript

```
const http = require("http");

const server = http.createServer((req, clientRes) => {

  http.get("http://example.com/api/users", (apiRes) => {

    let data = "";

    apiRes.on("data", (chunk) => {
      data += chunk;
    });

    apiRes.on("end", () => {

      clientRes.write(data);

      clientRes.end();

    });

  });

});
```



## 10. `res.write()` vs `res.end()`

Bahut important:

`res.write()`

Response ka data likhta/bhejta hai:

⌄ JavaScript

```
res.write("Hello");  
res.write(" World");
```

`res.end()`

Response complete karta hai:

⌄ JavaScript

```
res.end();
```

Example:

```
</> JavaScript  
  
res.write("Hello");  
res.write("Tarun");  
res.end();
```

Client ko:

HelloTarun

milega.

`res.end("Hello")` bhi kar sakte ho:

```
</> JavaScript  
  
res.end("Hello");
```

## 11. `res.write()` multiple times kyu?

HTTP response ko stream ki tarah bhej sakte ho:

`</>` JavaScript

```
res.write("Part 1");  
res.write("Part 2");  
res.write("Part 3");  
res.end();
```

Conceptually:

Server

↓

Part 1

↓

Part 2

↓

Part 3

↓

END



## 12. Promise + async/await ke saath?

Yahan ek important point hai:

`http.get()` callback-based API hai.

Isliye directly:

```
</> JavaScript  
  
await http.get(...)
```

normally nahi kar sakte, kyunki `http.get()` Promise return nahi karta.

Agar Promise style chahiye to `http.get()` ko Promise me wrap kar sakte ho:

```
</> JavaScript  
  
const http = require("http");  
  
function getData(url) {  
  
    return new Promise((resolve, reject) => {  
        ↓  
    })  
}
```

```
return new Promise((resolve, reject) => {  
  
  http.get(url, (res) => {  
  
    let data = "";  
  
    res.on("data", chunk => {  
      data += chunk;  
    });  
  
    res.on("end", () => {  
      resolve(data);  
    });  
  
  }).on("error", reject);  
  
});
```

Ab:

**</> JavaScript**

```
async function main() {  
  
    try {  
  
        const data = await getData(  
            "http://example.com/api/users"  
        );  
  
        console.log(data);  
  
    } catch (error) {  
  
        console.log(error);  
  
    }  
  
}  
  
main();
```

## 13. http.get vs Axios

Tum jo Axios padh rahe the, usse compare karo:

### Node http.get

</> JavaScript

```
http.get(url, (res) => {  
  
  let data = "";  
  
  res.on("data", chunk => {  
    data += chunk;  
  });  
  
  res.on("end", () => {  
    console.log(data);  
  });  
  
});
```

## Axios

`</>` JavaScript

```
const response = await axios.get(url);  
  
console.log(response.data);
```

Isliye Axios convenient hai.

Node http  
↓  
low-level  
↓  
callbacks / streams  
↓  
more manual work

Axios  
↓  
higher-level  
↓  
Promise